P1643R1
Add wait/notify to atomic_ref<T>

Published Proposal, 2019-07-19

Author:
David Olsen (NVIDIA)

Source:
GitHub

Issue Tracking:
GitHub

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:
SG1, LEWG, LWG

Table of Contents

§ 1. Introduction

[P1135] added the member functions `wait`, `notify_one`, and `notify_all` to `atomic<T>`, but did not add those same member functions to `atomic_ref<T>` due in part to scheduling concerns. This paper takes care of that, bringing the interface of `atomic_ref<T>` back in line with that of `atomic<T>`.

§ 2. Changelog

Revision 0: Initial version.

Revision 1: Fix formatting typo. Fix misspelling: "memory_orger" to "memory_order". Improved the wording of the *Remarks* clauses of `notify_one` and `notify_all` without changing the meaning.

§ 3. Wording

> Note: The following changes are relative to the post Kona 2019 working draft of ISO/IEC 14882, ([N4810]), with the changes from [P1135R5] merged in.

Modify the section about atomic waiting and notifying operations, [**atomics.wait**], which comes from P1135 not from N4810, as follows:

31.🔲 Waiting and notifying **[atomics.wait]**

1 ***Atomic waiting operations*** and ***atomic notifying operations*** provide a mechanism to wait for the value of an atomic object to change more efficiently than can be achieved with polling. Atomic waiting operations may block until they are unblocked by atomic notifying operations, according to each function's effects. [*Note:* Programs are not guaranteed to observe transient atomic values, an issue known as the A-B-A problem, resulting in continued blocking if a condition is only temporarily met. – *end note*]

2 [*Note*: The following functions are atomic waiting operations:

(2.1) — `atomic<T>::wait`.

(2.2) — `atomic_flag::wait`.

(2.3) — `atomic_wait` and `atomic_wait_explicit`.

(2.4) — `atomic_flag_wait` and `atomic_flag_wait_explicit`.

(2.5) — `atomic_ref<T>::wait`.

- *end note*]

3 [*Note*: The following functions are atomic notifying operations:

(3.1) — `atomic<T>::notify_one` and `atomic<T>::notify_all`.

(3.2) — `atomic_flag::notify_one` and `atomic_flag::notify_all`.

(3.3) — `atomic_notify_one` and `atomic_notify_all`.

(3.4) — `atomic_flag_notify_one` and `atomic_flag_notify_all`.

(3.5) — `atomic_ref<T>::notify_one` and `atomic_ref<T>::notify_all`.

- *end note*]

4 A call to an atomic waiting operation on an atomic object `M` is ***eligible to be unblocked*** by a call to an atomic notifying operation on `M` if there exist side effects `X` and `Y` on `M` such that:

(4.1) — the atomic waiting operation has blocked after observing the result of `X`,

(4.2) — `X` precedes `Y` in the modification order of `M`, and

(4.3) — `Y` happens before the call to the atomic notifying operation.

Modify the class synopsis for `atomic_ref` in **[atomics.ref.generic]** as follows:

31.6 Class template `atomic_ref` **[atomics.ref.generic]**

```
namespace std {
  template <class T> struct atomic_ref {

    // ...

    bool compare_exchange_strong(T&, T,
                    memory_order = memory_order_seq_cst) const noexcept;
    void wait(T, memory_order = memory_order::seq_cst) const noexcept;
    void notify_one() noexcept;
    void notify_all() noexcept;
  };
}
```

Add the following to the end of [**atomics.ref.operations**]:

```
void wait(T old, memory_order order = memory_order::seq_cst) const noexce
```

1 *Expects*: `order` is neither `memory_order::release` nor `memory_order::acq_rel`.

2 *Effects*: Repeatedly performs the following steps, in order:

(2.1) — Evaluates `load(order)` and compares its value representation for equality against that of `old`.

(2.2) — If they compare unequal, returns.

(2.3) — Blocks until it is unblocked by an atomic notifying operation or is unblocked spuriously.

3 *Remarks*: This function is an atomic waiting operation ([**atomics.wait**]) on atomic object `*ptr`.

```
void notify_one() noexcept;
```

4 *Effects*: Unblocks the execution of at least one atomic waiting operation on `*ptr` that is eligible to be unblocked ([**atomics.wait**]) by this call, if any such atomic waiting operations exist.

5 *Remarks*: This function is an atomic notifying operation ([**atomics.wait**]) on atomic object `*ptr`.

```
void notify_all() noexcept;
```

6 *Effects*: Unblocks the execution of all atomic waiting operations on `*ptr` that are eligible to be unblocked ([**atomics.wait**]) by this call.

7 *Remarks*: This function is an atomic notifying operation ([**atomics.wait**]) on atomic object `*ptr`.

Modify the class synopsis for the `atomic_ref` specialization for integral types in [**atomics.ref.int**] as follows:

```
namespace std {
  template <> struct atomic_ref<integral> {

    // ...

    bool compare_exchange_strong(integral&, integral,
                  memory_order = memory_order_seq_cst) const noexcept;
    void wait(integral, memory_order = memory_order::seq_cst) const noexc
    void notify_one() noexcept;
    void notify_all() noexcept;

    integral fetch_add(integral,
                  memory_order = memory_order_seq_cst) const noexcept;
    // ...
```

Modify the class synopsis for the `atomic_ref` specialization for floating-point types in **[atomics.ref.float]** as follows:

```
namespace std {
  template <> struct atomic_ref<floating-point> {

    // ...

    bool compare_exchange_strong(floating-point&, floating-point,
                  memory_order = memory_order_seq_cst) const noexcept;
    void wait(floating-point, memory_order = memory_order::seq_cst) const
    void notify_one() noexcept;
    void notify_all() noexcept;

    floating-point fetch_add(floating-point,
                  memory_order = memory_order_seq_cst) const noexcept;
    // ...
```

Modify the class synopsis for the `atomic_ref` partial specialization for pointer types in **[atomics.ref.pointer]** as follows:

```
namespace std {
  template <class T> struct atomic_ref<T*> {

    // ...

    bool compare_exchange_strong(T*&, T*,
                    memory_order = memory_order_seq_cst) const noexcept;
    void wait(T*, memory_order = memory_order::seq_cst) const noexcept;
    void notify_one() noexcept;
    void notify_all() noexcept;

    T* fetch_add(difference_type, memory_order = memory_order_seq_cst) cc
    // ...
```

§ Index

§ Terms defined by this specification

§ References

§ Informative References

[N4810]

Richard Smith. Working Draft, Standard for Programming Language C++. 15 March 2019. URL: https://wg21.link/n4810